



基于深度学习的光流估计(五)：轻量化架构



守护路路的星河

11 人赞同了该文章 >

收起

1.背景介绍

光流估计是计算机视觉中的一个重要任务，用于分析连续图像帧之间的像素运动。随着深度学习技术的发展，许多基于深度学习的光流估计方法已经提出，但这些方法往往计算量大、模型复杂，难以在资源受限的设备上实现实时处理，因此轻量化光流估计成为了研究的一个热点方向。

FlowNet2作为2018年前的SOTA，需要超过160M个参数才能实现准确的光流估计。[SpyNet+](#)通过在每个金字塔层中采用Warp来构建一个size较小的网络，参数为1.2M，但精度只能与FlowNet相匹配，而不能与FlowNet2相匹配。为对大位移场景保持较高的精度，特别是对快速移动的小物体，[RAFT+](#)构建了4D相关性volume和GRU模块⁺，但在预测高分辨率光流时会遇到内存和性能问题。

本文主要介绍一些针对光流估计中常见的网络结构、cost volume相关性计算等的性能优化方法，实现光流估计轻量化。

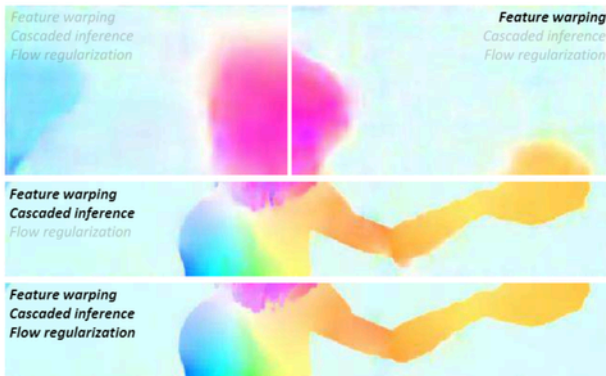
无关风月：基于深度学习的光流估计(二)：
baseline架构

8 赞同 · 0 评论 文章



2.模型架构优化方案

LiteFlowNet系列是2018提出的轻量级光流估计网络，这个网络和PWCNet有很多相似之处：包括图像金字塔和Cost Volumes⁺计算。其特点是参数量比较小，约为FlowNet2.0的0.03倍，运行速度快1.36倍。在随后文章作者做了一系列rethink，并提出了改良的LiteFlowNet2和3。



2.1 LiteFlowNet1

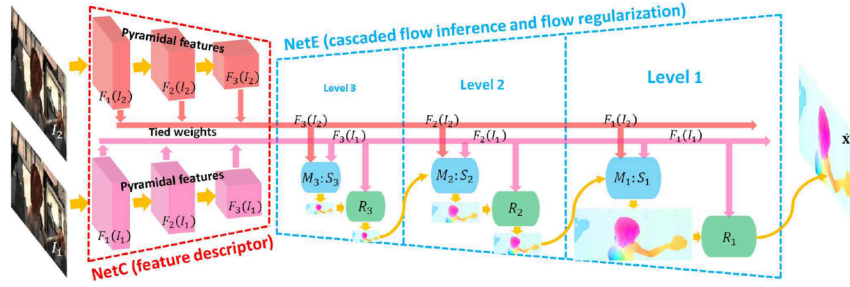
改进FlowNet2和SPyNet的轻量化设计有两个基本原则：

第一个是金字塔特征提取，LiteFlowNet由一个编码器和一个解码器组成，编码器将给定的图像对分别映射成两个多尺度高维特征金字塔，解码器在一个coarse-to-fine的框架中估计光流，每个金字塔层解码器通过选择和使用特征金字塔中相同分辨率的特征推理光流。与U-Net架构进行光流流量推断的FlowNet2相比，这种设计导致了更轻量级的网络。与SPyNet相比，LiteFlowNet分离了特征提取和光流估计的过程，这有助于更好地平衡效果和模型size。

第二个原则是对提取特征warp替代图像warp，FlowNet2和SPyNet使用之前的光流估计将第二张图像warp到第一张图像，然后使用warp的图像和第一张图像继续生成的特征迭代优化光流。这两



减少，因此可以使用更短的距离来建立Cost Volumes。此外仅在采样位置进行match，有效减少计算负担。

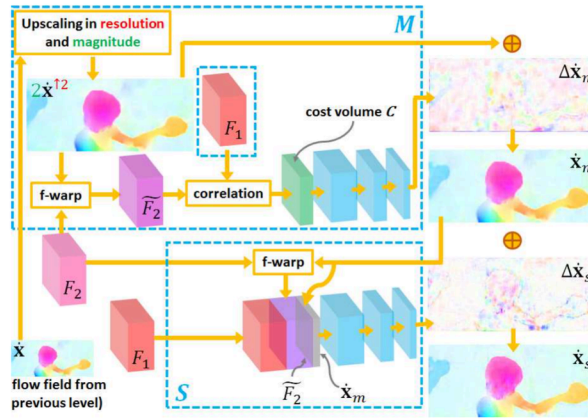


总的来说LiteFlowNet由两个紧凑的子网络组成，专门用于金字塔特征提取和光流估计。特征映射的空间维度在特征提取中是收缩的，光流场的空间维度在光流估计中是扩张的，我们将这两个子网络分别称为NetC和NetE。

NetC将任意给定的图像对转换成两个多尺度高维特征的金字塔，NetE由cascade光流推理和正则化模块组成，用于估计粗到细的光流场，下面主要介绍NetE：

Cascade光流inference

这是光流估计网络中最为经典的结构，通过在每层金字塔引入轻量级cascade结构，得到高效的光流推理。级联逐步提高光流精度，在不将更多错误传递给下一级的情况下对估计进行早期修正，并且允许网络结构中特征匹配的无缝合并。



经典的cascade模块M:S架构如上：在特征相关性匹配单元M中，f-warp获取在前一个金字塔层估计的上采样光流 $\mathbf{x}$ ，将帧 I_2 的高级特征 F_2 warp为 $\tilde{F}_2$ ，送入cost volume更新得到的粗光流 $\mathbf{x}_m$ ：

$$c(\mathbf{x}, \mathbf{d}) = \mathcal{F}_1(\mathbf{x}) \cdot \mathcal{F}_2(\mathbf{x} + \mathbf{d})/N,$$

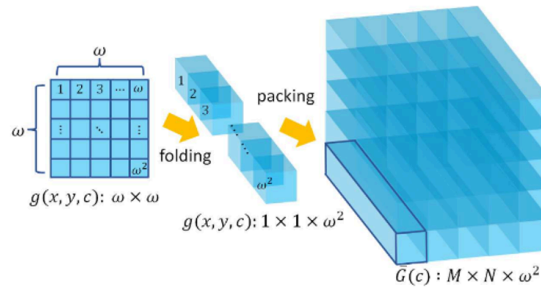
$$\dot{\mathbf{x}}_m = \underbrace{M(C(\mathcal{F}_1, \tilde{\mathcal{F}}_2; \mathbf{d}))}_{\Delta \dot{\mathbf{x}}_m} + s \dot{\mathbf{x}}^{\uparrow s}.$$

由于M是通过逐像素的相关性来聚合，因此光流估计 $\mathbf{x}_m$ 只能达到像素级的精度。在M之后引入第二个光流推理 $\mathbf{x}_m$ ，将像素级流场 $\mathbf{x}_m$ 细化到亚像素级精度，可以防止错误流被上采样放大并传递到下一个金字塔级别。亚像素级单元S通过计算剩余流量 $\Delta \mathbf{x}_s$ ，使 F_1 与 F_2 之间的特征空间距离最小化，得到更精确的光流场 $\mathbf{x}_s$ ：

$$\dot{\mathbf{x}}_s = \underbrace{S(\mathcal{F}_1, \tilde{\mathcal{F}}_2, \dot{\mathbf{x}}_m)}_{\Delta \dot{\mathbf{x}}_s} + \dot{\mathbf{x}}_m.$$

光流正则化

一问题，通常采用局部光流一致性和光流边界与强度边缘的共现性约束。



f-lcon的操作很好地受到像素值扩散的拉普拉斯公式的支配，与传统CNN中使用的局部卷积(lcon)相比f-lcon更加一般化。该算法不仅对特征映射的每个位置使用不同的滤波器，而且对单个块自适应构建滤波器。考虑一个一般的情况，一个必须正则化的向量值特征F有C个通道和一个空间维度M × N，定义 $\mathbf{G} = \{\mathbf{G}_i\}$ 作为F-lcon层中使用的滤波器集合。F-lcon对F的运算可表示为：

$$f_g(x, y, c) = g(x, y, c) * f(x, y, c)$$

f-lcon滤波器专门用于平滑光流，如果光流在patch上的变化是平滑的，它应该表现为一个平均滤波器，而光流在边界上则不能过于平滑。因此定义了一个特征驱动的CNN距离度量D，它使用金字塔特征 $\mathbf{F}_1$ 、光流 $\mathbf{x}_s$ 、遮挡概率 $\mathbf{O}$ 来估计局部光流平滑性：

$$\mathcal{D} = R_D(\mathcal{F}_1, \dot{\mathbf{x}}_s, \mathbf{O}).$$

$$g(x, y, c) = \frac{\exp(-\mathcal{D}(x, y, c)^2)}{\sum_{(x_i, y_i) \in N(x, y)} \exp(-\mathcal{D}(x_i, y_i, c)^2)},$$

N表示当前点w*w的邻域，这里提供了一种有效执行f-lcon的机制：对于C通道输入F，我们使用C张量G(1), ..., G(C)来存储f-lcon滤波器集G，每个f-lcon滤波器G被折叠成一个1 * 1 * w * w的三维列，然后打包到一个M * N * w的三维张量G(C)的(x,y)项中。同样的折叠操作也应用于F的每个通道中的每个patch，得到C张量F(1), ..., F(C)，则最开始的公式可改写为点乘形式：

$$F_g(c) = \bar{G}(c) \odot \bar{F}(c),$$

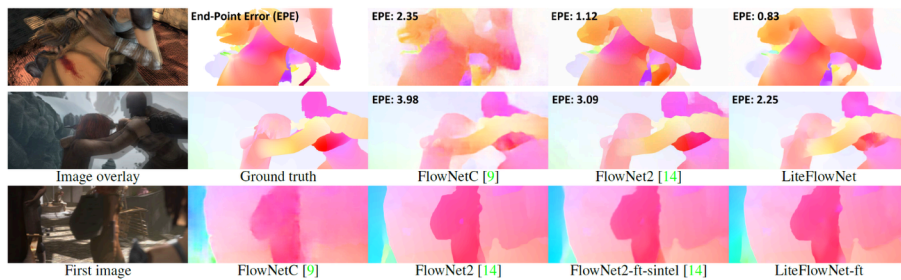
最终的实验中对比FlowNet2和SpyNet，LiteFlowNet在参数量和runtime有大幅领先：

Model	Shallow		Deep	Very Deep	
	FlowNetC	SPyNet	LFlowNetX	LFlowNet	FlowNet2
# layers	26	35	74	99	115
# param. (M)	39.16	1.20	0.90	5.37	162.49
Runtime (ms)	32.28	(129.83)	35.83	90.25	122.39

在平均EPE的统计中，LiteFlowNet的效果整体也没有太大的下降：

Conventional	DeepFlow ¹ [31]	2.66	5.38	3.57	7.21	4.48	5.8	10.63	26.52%	29.18%	0.25	0.42
	Classic+NLP [27]	4.49	6.73	7.46	8.29	-	7.2	-	-	-	0.22	0.32
	PCA-Layers ¹ [33]	3.22	5.73	4.52	7.89	5.99	5.2	12.74	27.26%	-	0.66	-
	EpicFlow ¹ [22]	2.27	4.12	3.56	6.29	3.09	3.8	9.27	27.18%	27.10%	0.31	0.39
	FlowFields ¹ [1]	1.86	3.75	3.06	5.81	3.33	3.5	8.33	24.43%	-	0.27	0.33
Hybrid	Deep DiscreteFlow [11]	-	3.86	-	5.73	-	3.4	-	-	21.17%	-	-
	Bailer <i>et al.</i> [2]	-	3.78	-	5.36	-	3.0	-	-	19.44%	-	-
	DC Flow [34]	-	-	-	5.12	-	-	-	-	14.86%	-	-
Heavyweight CNN	FlowNetS [9]	4.50	7.42	5.45	8.43	8.26	-	-	-	-	1.09	-
	FlowNetS-ft [9]	(3.66)	6.96	(4.44)	7.76	7.52	9.1	-	-	-	0.98	-
	FlowNetC [9]	4.31	7.28	5.87	8.81	9.35	-	-	-	-	1.15	-
	FlowNetC-ft [9]	(3.78)	6.85	(5.28)	8.51	8.79	-	-	-	-	0.93	-
	FlowNet2-S ³ [14]	3.79	-	4.99	-	7.26	-	14.28	51.06%	-	1.04	-
	FlowNet2-S re-trained ²	3.96	-	5.37	-	7.31	-	14.51	51.38%	-	1.13	-
	FlowNet2-C ³ [14]	3.04	-	4.60	-	5.79	-	11.49	44.09%	-	0.98	-
	FlowNet2 [14]	2.02	3.96	3.54 ⁴	6.02	4.01 ⁵	-	10.08 ⁵	29.99% ⁵	-	0.35	0.52
	FlowNet2-ft-sintel [14]	(1.45)	4.16	(2.19 ⁴)	5.74	3.54 ⁵	-	9.94 ⁵	28.02% ⁵	-	0.35	-
	FlowNet2-ft-kitti [14]	3.43	-	4.83 ⁴	-	(1.43 ⁵)	1.8	(2.36 ⁵)	(8.88% ⁵)	11.48%	0.56	-
Lightweight CNN	SPyNet [21]	4.12	6.69	5.57	8.43	9.12	-	-	-	-	0.33	0.58
	SPyNet-ft [21]	(3.17)	6.64	(4.32)	8.36	3.36 ⁶	4.1	-	-	35.07%	0.33	0.58
	LiteFlowNetX-pre	3.70	-	4.82	-	6.81	-	16.64	36.64%	-	0.45	-
	LiteFlowNetX	3.58	-	4.79	-	6.38	-	15.81	34.90%	-	0.46	-
	LiteFlowNet-pre	2.78	-	4.17	-	4.56	-	11.58	32.59%	-	0.45	-
	LiteFlowNet	2.52	-	4.05	-	4.25	-	10.46	29.30%	-	0.39	-
	LiteFlowNet-ft	(1.64)	4.86	(2.23)	6.09	(1.26)	1.7	(2.16)	(8.16%)	10.24%	0.33	0.50

在Sintel测试集⁺上的表现：



消融实验证明了LiteFlowNet各个组件的优势：

Variants	M	MS	WM	WSR	WMS	ALL
Feature Warping	✗	✗	✓	✓	✓	✓
Descriptor Matching	✓	✓	✓	✗	✓	✓
Sub-pix. Refinement	✗	✓	✗	✓	✓	✓
Regularization	✗	✗	✗	✓	✗	✓
FlyingChairs (train)	3.75	2.70	2.98	1.63	1.82	1.57
Sintel clean (train)	4.70	4.17	3.54	3.19	2.90	2.78
Sintel final (train)	5.69	5.30	4.81	4.63	4.45	4.17
KITTI12 (train)	9.22	8.01	6.17	5.03	4.83	4.56
KITTI15 (train)	18.24	16.19	14.52	13.20	12.32	11.58



2.2 LiteFlowNet2

LiteFlowNet2主要是在LiteFlowNet的基础上做了深入的原理研究，以及网络结构参数的一些小调整：

Deep光流和传统变分法对比

传统变分的光流估计方法中，对于输入 I_1 和 I_2 假设光流为 $\mathbf{u}$ ，可以通过如下约束求解光流：

$$\begin{aligned}
 E(\mathbf{u}) &= E_{dat}(\mathbf{u}) + \lambda E_{reg}(\nabla \mathbf{u}) \\
 &= \int_{\Omega} (e_{data}(\mathbf{u}) + \lambda e_{reg}(\nabla \mathbf{u})) d\mathbf{x},
 \end{aligned}$$

其中 E_{data} 是数据保真项， E_{reg} 是正则化项， w 是平滑参数：

在保真项中 D_i 主要是表明亮度、梯度恒定的对齐状态函数，另外 D_i 需要保持数据项和正则化项的可微性，因为能量方程通常需要使用欧拉-拉格朗日方程求解。与传统方法相比，最先进的光流网络没有明确定义那些匹配量。无监督方法使用的光度损失计算为第一张图像和扭曲的第二张图像之间的差异、SPyNet使用来自图像金字塔的一对图像在相应的金字塔级别生成光流场，PWCNet和LiteFlowNet使用可学习的特征编码器。

更详细地说，LiteFlowNet的NetC训练金字塔型特征描述符，它将给定的图像对 I_1 和 I_2 分别转化为两个多尺度高维特征金字塔。随着特征描述符的引入训练了NetE中的级联流推理，通过计算高级特征 F_1 和 F_2 之间的对应关系来解决它们之间差异的最小化问题，换句话说LiteFlowNet和其他光流CNN中使用的特征编码器类似于变分方法中数据项的作用。

而单纯依靠数据保真度计算的光流场容易受到异常值的影响，能量泛函通常被增强以加强全局的平滑性。向量场的正则化可以看作是像素值的扩散，将欧拉-拉格朗日方程应用于最开始的能量函数，正则化分量由下式给出：

$$\operatorname{div}(\partial_{\nabla \mathbf{u}} E_{reg}(\nabla \mathbf{u})) = \operatorname{div}(\mathbf{D} \nabla \mathbf{u}),$$

$\mathbf{D}$ 是2X2的kernel，进一步可以转化为：

$$\operatorname{div}(\mathbf{D} \nabla \mathbf{u}) = \operatorname{trace}(\mathbf{T} \mathbf{H}_i), i = 1, 2,$$

上式的解为：

$$\mathbf{u} = K(\mathbf{T}) * \mathbf{u}',$$

可以看到对光流场施加平滑性约束相当于对数据项生成的中间光流场进行具有二维方向高斯核的卷积。但与上式中需要工程正则化结构的平滑核不同，LiteFlowNet使用特征驱动的局部卷积(F-lconv)滤波器G，通过将F-lconv核适应于编码器产生的特征F、来自数据项的金字塔特征 F_1 、光流 $\mathbf{x}_s$ 、遮挡概率 O 来对光流场中的每个光流向量进行不同的正则化。

总结下来在data层面，LiteFlowNet的级联流场推理网络类似于变分光流方法中数据项的作用。仅仅由数据保真度计算的流场对于奇异值是非常敏感的，LiteFlowNet的特征驱动正则化项类似于变分方法中正则化项的作用。

CNN和基函数方法对比

图像位移量的参数化模式可以通过基函数来表示：

$$\mathbf{u}_{vec} = \sum_{i=1}^C a_i \mathbf{m}_i,$$

而LiteFlowNet中金字塔光流推理中，位移量可以表示为：

$$\begin{aligned} \mathcal{F}^{N-1} &= \sigma(\mathbf{W}^{N-1} * \mathcal{F}^{N-2} + \mathbf{b}^{N-1}), \\ \Delta \mathbf{u} &= \mathbf{W}^N * \mathcal{F}^{N-1} + \mathbf{b}^N, \end{aligned}$$

这里*表示卷积， N 代表卷积层数量。 $\mathbf{W}$ 和 $\mathbf{F}$ 分别表示第 i 层的卷积滤波器和特征映射，每个特征图都有一个可训练的偏置 $\mathbf{b}$ 。将特征的每个channel拆分出来，则可以表示为：

$$\Delta \mathbf{u} = \sum_{i=1}^C (\mathbf{W}_i * \mathcal{F}_i + b_i),$$

可以看到向量化的 $\mathbf{W} * \mathbf{F} + \mathbf{b}$ 类似于传统方法的 $\mathbf{am}$ ，即图像运动的参数化模型已经被良好地封装在文章所设计的级联流动参考中。

LiteFlow2相较于LiteFlow的改进

金字塔层数优化：如下表所示，NetE decoder 60%的时间在金字塔第三层到第二层的位置，而这里对光流AEE精度的提升却只有0.17，所以LiteFlow2只做了金字塔6-3层的解码。

	-	-	-28.8%	-21.3%	-12.5%	-6.4%
KITTI 2012	-	8.58	6.04	4.67	4.18	4.00
	-	-	-29.6%	-22.7%	-10.5%	-4.4%
Runtime (ms)	14.36	1.69	2.03	4.88	13.06	52.51
	-	-	+20.1%	+140%	+168%	+302%

通道数增加：为了补偿损失金字塔层数减少的损失，我们在NetE的cascade inference中为每个解码器在128通道和64通道卷积层之间添加了两个卷积层(具有128和96个输出通道)。

伪光流推理与正则化：分别去除了原始光流推理和正则化中最后一层之前的所有层。然后，我们在第3层的光流推理和正则化中，分别从最后一层之前的层中用上采样特征来替换被删除的层。这个操作让第2层的运行时间大大减少了：从52.51ms降至8.95ms。伪推理可以提高KITTI测试集⁺上的精度，但在sintel测试集上没有明显的提高。由于后者是一个合成数据集符合预期，而在KITTI真实场景中上多进行一次光流推理和正则化，有利于对之前的光流估计进行细化。

实验结果如下：

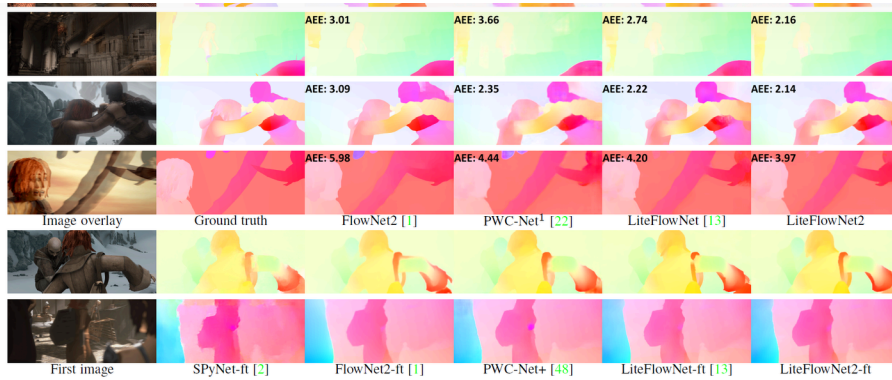
	Method	Sintel Clean		Sintel Final		KITTI 2012		KITTI 2015	
		Train	Test	Train	Test	Train	Test (Out-Noc)	Train (Fl-all)	Test (Fl-all)
Conventional	LDOF ¹ [8]	4.64	7.56	5.96	9.12	10.94	12.4	18.19	38.11%
	DeepFlow ¹ [24]	2.66	5.38	3.57	7.21	4.48	5.8	10.63	26.52%
	Classic+NLP [46]	4.49	6.73	7.46	8.29	-	7.2	-	-
	PCA-Layers ¹ [33]	3.22	5.73	4.52	7.89	5.99	5.2	12.74	27.26%
	EpicFlow ¹ [27]	2.27	4.12	3.56	6.29	3.09	3.8	9.27	27.18%
	FlowFields ¹ [40]	1.86	3.75	3.06	5.81	3.33	3.5	8.33	24.43%
Hybrid	Deep DiscreteFlow [38]	-	3.86	-	5.73	-	3.4	-	21.17%
	Bailei <i>et al.</i> [39]	-	3.78	-	5.36	-	3.0	-	19.44%
	DC Flow [47]	-	-	-	5.12	-	-	-	14.86%
Heavyweight CNN	FlowNetS [12]	4.50	7.42	5.45	8.43	8.26	-	-	-
	FlowNetS-ft [12]	(3.66)	6.96	(4.44)	7.76	7.52	9.1	-	-
	FlowNetC [12]	4.31	7.28	5.87	8.81	9.35	-	-	-
	FlowNetC-ft [12]	(3.78)	6.85	(5.28)	8.51	8.79	-	-	-
	FlowNet2-S ² [1]	3.79	-	4.99	-	7.26	-	14.28	51.06%
	FlowNet2-C ² [1]	3.04	-	4.60	-	5.79	-	11.49	44.09%
	FlowNet2 [1]	2.02	3.96	3.54³	6.02	4.01 ⁴	-	10.08 ⁴	29.99% ⁴
	FlowNet2-ft-sintel [1]	(1.45)	4.16	(2.19 ³)	5.74	3.54⁴	-	9.94⁴	28.02%⁴
	FlowNet2-ft-kitti [1]	3.43	-	4.83 ³	-	(1.43 ⁴)	1.8	(2.36 ⁴)	(8.88% ⁴)
									11.48%
Lightweight CNN	SPyNet [2]	4.12	6.69	5.57	8.43	9.12	-	-	-
	SPyNet-ft [2]	(3.17)	6.64	(4.32)	8.36	3.36 ⁶	4.1	-	35.07%
	PWC-Net [22]	2.55	-	3.93	-	4.14	-	10.35	33.67%
	PWC-Net-ft [22]	(2.02)	4.39	(2.08)	5.04	(1.45)	1.7	(2.16)	(9.80%)
	PWC-Net-ROB [48]	(1.81)	3.90	(2.29)	4.90	-	-	-	9.60%
	PWC-Net-ft+ [48]	(1.71)	3.45	(2.34)	4.60	(0.99)	1.4	(1.47)	(7.59%)
	LiteFlowNetX-pre [13]	3.70	-	4.82	-	6.81	-	16.64	36.64%
	LiteFlowNetX [13]	3.58	-	4.79	-	6.38	-	15.81	34.90%
	LiteFlowNet-pre [13]	2.78	-	4.17	-	4.56	-	11.58	32.59%
	LiteFlowNet ⁷ [13]	2.48	-	4.04	-	4.00	-	10.39	28.50%
	LiteFlowNet-ft ⁷ [13]	(1.35)	4.54	(1.78)	5.38	(1.05)	1.6	(1.62)	(5.58%)
	LiteFlowNet2-pre	2.78	-	4.14	-	4.11	-	11.31	32.12%
	LiteFlowNet2	2.32	-	3.85	-	3.77	-	9.83	28.45%
	LiteFlowNet2-ft	(1.41)	3.48	(1.83)	4.69	(0.95)	1.4	(1.33)	(4.32%)

在非遮挡区域中端点误差(EPE) >3个像素的错误像素百分比与Fl-all:在所有像素上平均的离群值百分比

轻量化的几个操作对效果的影响：

	[13]	LiteFlowNet2		
Flow levels up to level 3	✗	✓	✗	✗
Flow levels up to (pseudo) ¹ level 2	✓	✗	✓	✓
Double GT resolution at each level	✗	✗	✗	✓
KITTI 2012: Train	(1.05)	(1.07)	(1.00)	(0.95)
Test (Out-Noc)	3.27%	3.07%	2.72%	2.63%
Test (Out-all)	7.27%	6.92%	6.30%	6.16%
Test (Avg-all)	1.6	1.5	1.4	1.4
KITTI 2015: Train	(1.62)	(1.61)	(1.47)	(1.33)
Train (Fl-all)	(5.58%)	(5.57%)	(4.80%)	(4.32%)
Test (Fl-bg)	9.66%	8.72%	7.85%	7.62%
Test (Fl-fg)	7.99%	8.20%	7.20%	7.64%
Test (Fl-all)	9.38%	8.63%	7.74%	7.62%

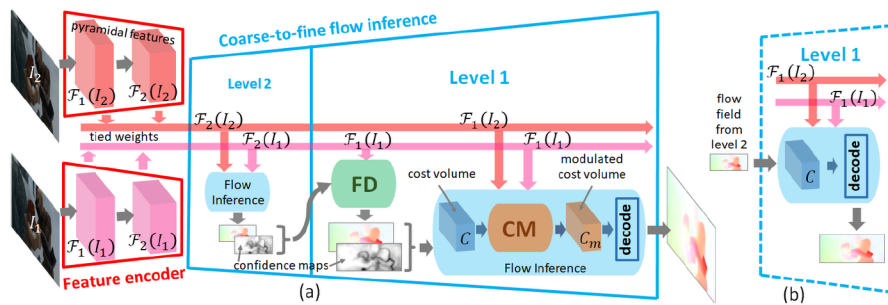
不同方法在Sintel训练集上的结果，精细的细节得到了很好的保留，在LiteFlowNet2和LiteFlowNet2-ft的光流map中可以观察到较少的伪像。



2.3 LiteFlowNet3

深度学习图像对齐领域的成功依赖于cost volume和coarse-to-fine的两大策略，目前遗留的问题主要是当图像中存在部分遮挡或重复纹理区域时，feature matching问题就变得ill-posed（一对多、一对无）。这将导致cost volume包含异常值，并影响从它进行的光流解码。此外coarse-to-fine的光流推理还需要精确的光流初始值，错误的光流初值会影响后续推理。

为了解决上述挑战，开发了两个专门的CNN模块:Cost Volume Modulation(CM)和Flow Field Deformation(FD)，得到的网络被命名为LiteFlowNet3。

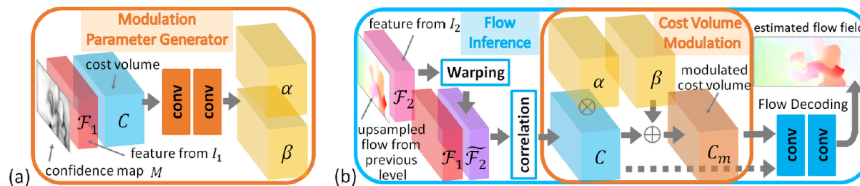


LiteFlowNet3:(FD)和(CM)以及置信度map被合并到LiteFlowNet3中，为了便于展示这里只展示了一个2级编码器解码器结构。(b) LiteFlowNet2中的光流推理。

Cost Volume Modulation

特征匹配中不准确的异常相关性向量需要被修正，为了处理遮挡，早期的工作使用以offset-centered matching windows来改进匹配过程，在解码之前对cost volume进行过滤以去除异常值。然而现有的光流CNN使用卷积从cost volume推理光流，但没有明确解决异常值问题。

本模块使用了自适应仿射变换来修正cost volume中的每个vector，以及置信度mask确定不可靠的位置。

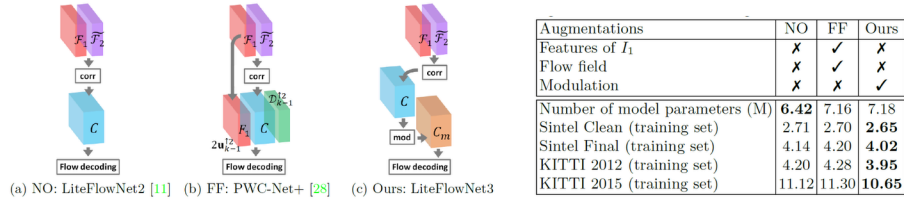


设计了一个通过自适应调制来过滤cost volumes中的异常值的Cost Volume Modulation，推理时在金字塔各层，C中的每个cost vector $c(\mathbf{x})$ 通过仿射变换 $(\alpha(\mathbf{x}), \beta(\mathbf{x}))$ 自适应调制如下：

$$c_m(\mathbf{x}) = \alpha(\mathbf{x}) \otimes c(\mathbf{x}) \oplus \beta(\mathbf{x}),$$

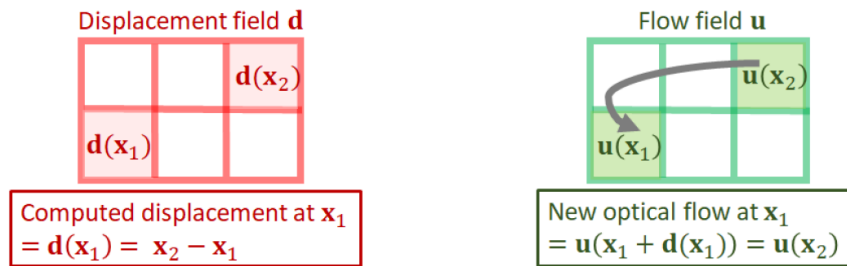
而参数生成如图a所示，引入置信度图M是为了方便调制参数的生成。具体来说， $M(\mathbf{x})$ 精确指出了光流场中在 $\mathbf{x}$ 处具有精确光流的概率。该置信度图是通过在前面的光流解码器中引入额外的输出来构建的，使用sigmoid函数将其值约束为[0,1]。我们使用一个 L_2 损失来训练置信图，GT标签为 $M_{gt}(\mathbf{x})$ ，如下所示：

LiteFlowNet3对比LiteFlow2和PWC的cost volume处理:

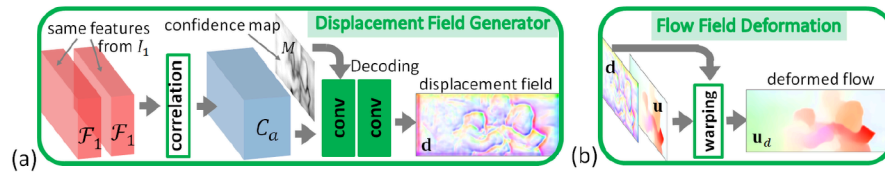


Flow Field Deformation

如下图所示，假设 $u(x_1)$ 根据先验被判定为不准确的光流值，而根据置信度map $u(x_2)$ 是精确的光流值。当 I_1 的特征 $F(x_1)$ 和 $F(x_2)$ 是相似的时候，我们就会用 $u(x_2)$ 替换 $u(x_1)$ 。



前面的分析只是针对单个点，为了覆盖整个场，我们需要为光流场中的每个位置找到一个位移向量。也就是说，我们需要有一个位移场来指导流场的meta-warp，如下图b所示：



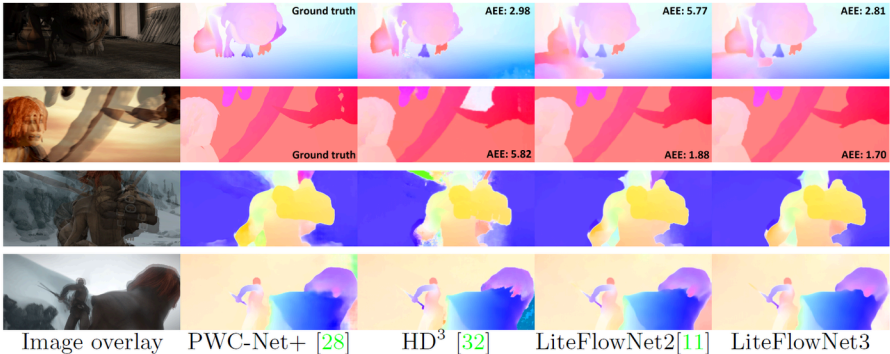
为了生成位移场，需要找到与目标图像点具有相似特征的图像点的位置，这是通过对自相关代价量进行decoding来完成的。

最终实验结果如下，在公开数据集上的指标：

Method	Sintel Clean		Sintel Final		KITTI 2012				KITTI 2015			
	train	test	train	test	train	test	(All)	test (Noc)	train	train (Fl-all)	test (Fl-fg)	test (Fl-all)
FlowNetS [6]	(3.66)	6.96	(4.44)	7.76	7.52	9.1	-	-	-	-	-	-
FlowNetC [6]	(3.78)	6.85	(5.28)	8.51	8.79	-	-	-	-	-	-	-
FlowNet2 [13]	(1.45)	4.16	(2.19)	5.74	(1.43)	1.8	1.0	(2.36)	(8.88%)	8.75%	11.48%	
FlowNet3 [14]	(1.47)	4.35	(2.12)	5.67	(1.19)	-	-	(1.79)	-	-	8.60%	
SPyNet [24]	(3.17)	6.64	(4.32)	8.36	3.36	4.1	2.0	-	-	43.62%	35.07%	
Devon [19]	-	4.34	-	6.35	-	2.6	1.3	-	-	19.49%	14.31%	
PWC-Net [27]	(2.02)	4.39	(2.08)	5.04	(1.45)	1.7	0.9	(2.16)	(9.80%)	9.31%	9.60%	
PWC-Net+ [28]	(1.71)	3.45	(2.34)	4.60	(0.99)	<u>1.4</u>	<u>0.8</u>	(1.47)	(7.59%)	7.88%	7.72%	
IRR-PWC [12]	(1.92)	3.84	(2.51)	4.58	-	1.6	0.9	(1.63)	(5.32%)	<u>7.52%</u>	7.65%	
SENSE [16] [†]	(1.54)	3.60	(2.05)	4.86	(1.18)	1.5	-	(2.05)	(9.69%)	9.33%	8.16%	
HD ³ [32] [†]	(1.70)	4.79	(1.17)	4.67	(0.81)	1.4	0.7	(1.31)	(4.10%)	9.02%	6.55%	
SelfFlow [18] ^{†,‡}	(1.68)	3.75	(1.77)	4.26	(0.76)	1.5	0.9	(1.18)	-	12.48%	8.42%	
VCN-small [31]	(1.84)	3.26	(2.44)	4.73	-	-	-	(1.41)	(5.5%)	-	7.74%	
LiteFlowNet [10]	(1.35)	4.54	(1.78)	5.38	(1.05)	1.6	<u>0.8</u>	(1.62)	(5.58%)	7.99%	9.38%	
LiteFlowNet2 [11]	(1.30)	3.48	(1.62)	4.69	(0.95)	<u>1.4</u>	0.7	(1.33)	(4.32%)	7.64%	7.62%	
LiteFlowNet3	(1.32)	2.99	(1.76)	<u>4.45</u>	(0.91)	1.3	0.7	(1.26)	(3.82%)	7.75%	7.34%	
LiteFlowNet3-S	(1.43)	<u>3.03</u>	(1.90)	4.53	(0.94)	1.3	0.7	(1.39)	(4.35%)	6.96%	<u>7.22%</u>	

Sintel数据集上的表现：

FlowNet2 [13]	4.16	5.74	1.56	2.75	25.40	30.11
Devon [19]	4.34	6.35	1.74	3.23	25.58	31.78
PWC-Net+ [28]	3.45	4.60	1.41	2.25	20.12	23.70
IRR-PWC [12]	3.84	4.58	1.47	2.15	23.22	24.36
SENSE [16] [†]	3.60	4.86	1.38	2.30	21.75	25.73
HD ³ [32] [†]	4.79	4.67	1.62	2.17	30.63	24.99
LiteFlowNet2 [11]	3.48	4.69	1.33	2.25	20.64	24.57
LiteFlowNet3	2.99	4.45	1.15	2.09	18.08	23.68



消融实验结果如下：

Settings	NO	CM-	CMFD-	CM	CMFD
Cost Volume Modulation	✗	✓	✓	✓	✓
Flow Field Deformation	✗	✗	✓	✗	✓
Confidence map	✗	✗	✗	✓	✓
Sintel clean (training set)	2.78	2.66	2.63	2.65	2.59
Sintel final (training set)	4.14	4.09	4.06	4.02	3.91
KITTI 2012 (training set)	4.11	4.02	4.06	3.95	3.88
KITTI 2015 (training set)	11.31	11.01	10.97	10.65	10.40



性能方面，Intel Xeon E5 在Nvidia GTX1080上计算(1024 X 436)的pair，需要59ms和5.2M参数量。

3. 相关性计算优化方案

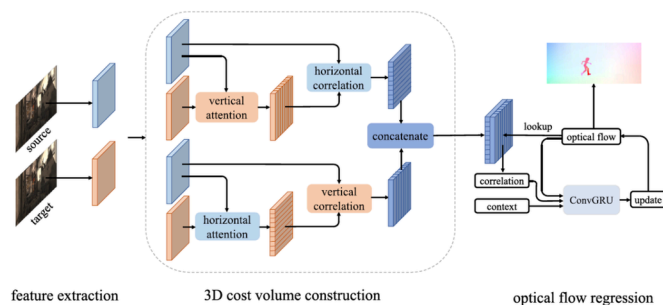
coarse-to-fine的网络架构构建特征金字塔，由粗到细的计算相关性pair，这种结构会丢失一些小位移物体。而如RAFT的全局相关性计算不需要逐层的特征warp但内存消耗巨大，并且4D相关性cost volume的计算性能开销也较大。因此对于cost volume相关性计算这个模块，发展了大量的优化方案。

3.1 Flow1D⁺

本文关键思想是将2D搜索分别在垂直和水平方向上分解为两个1D的替代，这样就可以使用1D相关性来构建紧凑的cost volume。直观地说，这种1D相关性对于光流估计是不够的，光流估计本质上是一个2D搜索问题。然而如果可以沿着与相关方向正交的方向传播target图像上的信息，则计算出的cost volume将包含source像素与其对应点之间有意义的相关性。这一想法促使为1D相关性设计合适的特征传播和聚合方案。



受transformer启发提出了用注意机制来学习这种传播，首先在source特征上应用1D self-attention，然后在source特征和target特征之间进行1D cross-attention。1D公式分别产生两个尺寸为 $(H \times W \times W)$ 和 $(H \times W \times H)$ 的3D cost volume，然后将它们concat起来用于随后的光流refine。通过这种方式，我们将all-pair相关的复杂性从 $H \times W \times H \times W$ 降低到 $H \times W \times (H + W)$ 。

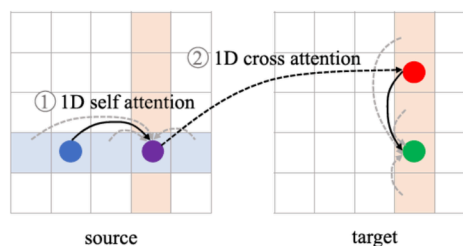


1D Attention

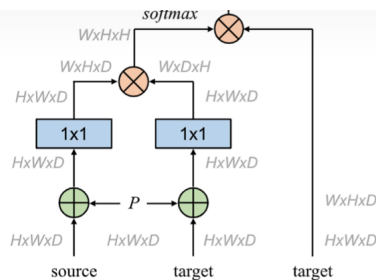
source和target特征分别为 F_1 和 F_2 ，shape为 $H \times W \times D$ ，我们的目标是生成一个新的feature F'_2 ，使得在相同列的所有特征向量互相感知：

$$\hat{F}_2(h, w) = \sum_{i=0}^{H-1} f_{i,h,w} F_2(i, w),$$

$F'_2(h, w)$ 代表点 (h, w) 的特征向量， f 是融合权重，这样每个点都会融合相同列特征信息。现在问题变成了 f 如何设置，去合适的聚合列信息。受transformer的启发，我们通过注意力机制去学习 f 。与计算all-pair相似度的原始attention不同，我们只以轴对齐的方式执行1D attention。如图所示，attention操作的目标是将红点(蓝点的对应)的信息传播到与蓝点位于同一行的绿色点上。



为了实现这一点，一种解决方案是在source和target的同一列之间应用1D cross attention操作，使target特征聚合依赖source特征。然而，source图像中的同一列可能不包含相应的像素(蓝点)，这使得学习适当的聚合变得困难。为了解决这个问题，我们首先执行一个1D self attention操作在source图像的水平方向，它将source图像上对应点的信息传播到整行。1D垂直方向cross交叉注意的计算如下图所示：



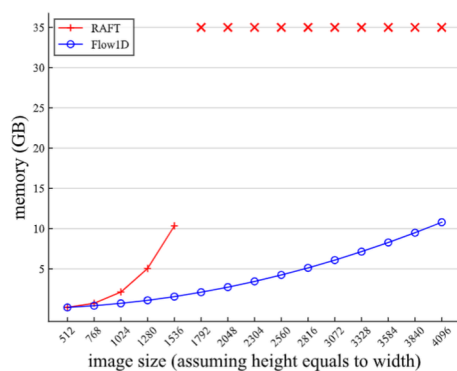
输入为source特征的self-attend和原始target特征，我们还在引入了位置信息 P ， P 是与DETR相同的固定2D位置编码。首先用1X1卷积将 $F_1 + P$ 与 $F_2 + P$ 投影到embedding空间，然后通过reshape获取 F_1 与 F_2 水平方向的attention矩阵：WxHxD, WxDxH，并对最后两维相乘得到 A ：WxHxD，将 A 与target feature相乘得到最后的 F'_2 。最后进行1D的cross attention操作， F'_2 中的每个位置都会encode同一列的位置信息。

1D correlation

当得到垂直方向的聚合特征 F'_2 ，在水平方向进行一个简单的1D搜索建立一个3D cost volume： $H \times W \times (2R+1)$ 。

$$C(h, w, R+r) = \frac{1}{\sqrt{D}} F_1(h, w) \cdot \hat{F}_2(h, w+r)$$

最终在内存上对比RAFT有着明显的优势：



在1080*1920的数据集上，Flow1D与RAFT效果持平但有着6倍的内存收益：

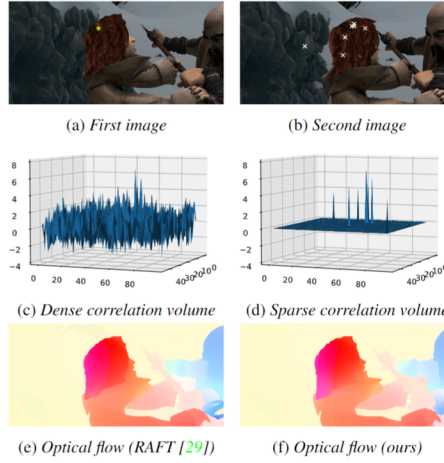
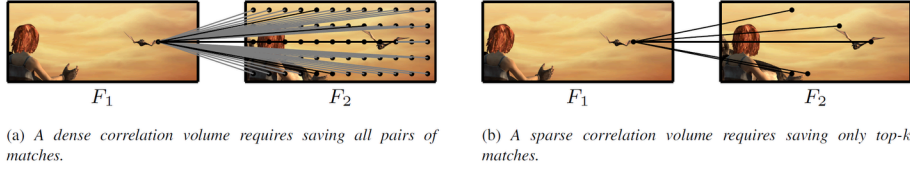


指标结果如下：

FlowNet2 [18]	(1.45)	(2.01)	4.16	5.74	(6.8)	11.48
PWC-Net+ [35]	(1.71)	(2.34)	3.45	4.60	(5.3)	7.72
HD ³ [45]	(1.87)	(1.17)	4.79	4.67	(4.1)	6.55
VCN [43]	(1.66)	(2.24)	2.81	4.40	(4.1)	6.30
MaskFlowNet [47]	-	-	2.52	4.17	-	6.10
RAFT [37]	(0.77)	(1.27)	1.61	2.86	(1.5)	5.10
Flow1D	(0.84)	(1.25)	2.24	3.81	(1.6)	6.27

3.2 SCV

对于单一图像的特性向量在其他图像上只有少量高度相似的特征向量，因此低相似度特征对于预测毫无贡献其浪费大量资源。



首先回顾一下cost volume的计算公式，特征 F_1 上 $\mathbf{x}$ 点和 F_2 上 $\mathbf{x} + \mathbf{d}$ 点之间的相关性表示为：

$$C(\mathbf{x}, \mathbf{d}) = F_1(\mathbf{x}) \cdot F_2(\mathbf{x} + \mathbf{d}).$$

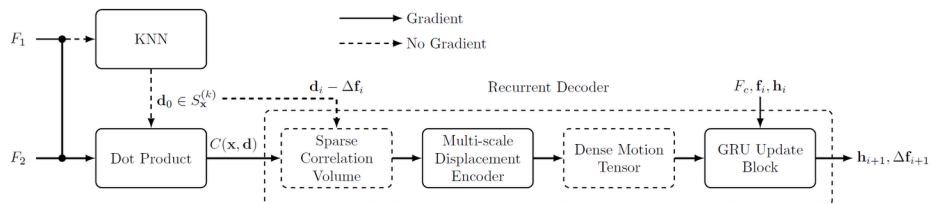
$$\mathcal{C} = \{C(\mathbf{x}, \mathbf{d}) \mid \mathbf{x} \in \mathcal{X}, \mathbf{d} \in \mathcal{D}\}.$$

这里集合 $\mathcal{C}$ 表示整个 F_1 上 $h \times w$ 的点和 F_2 上 $h \times w$ 个点之间的4D相关性表，基于上述volumn，定义一个新的稀疏表，这里针对每个 F_1 特征只在 F_2 上匹配 k 个tensor：

$$S_{\mathbf{x}}^{(k)} = \arg \max_{S \subset \mathcal{D}, |S|=k} \sum_{\mathbf{d} \in S} C(\mathbf{x}, \mathbf{d})$$

$$\tilde{\mathcal{C}} = \{C(\mathbf{x}, \mathbf{d}) \mid \mathbf{d} \in S_{\mathbf{x}}^{(k)}, \mathbf{x} \in \mathcal{X}\}.$$

这里 $\mathbf{x}$ 还是 $h \times w$ 个点，但 $\mathbf{x} + \mathbf{d}$ 变成了 $S_{\mathbf{x}}$ 对应的 k 个点。以此思想，我们的网络架构如下：

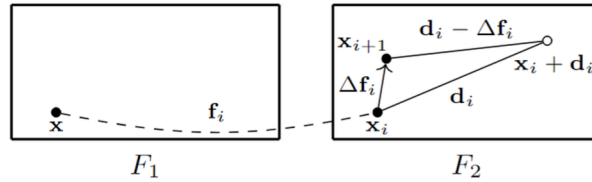


KNN

为了构建稀疏相关体积，使用最近邻(kNN)模块为 F_1 中的每个特征向量计算一组具有最大 k 个相关分数的index。通过取 F_1 中的每个特征向量与 F_2 中指标给出的前 k 个特征向量之间的点积来计算稀疏相关Volumn。

在反向传播期间，梯度只传播到由kNN模块选择的 k 个特征向量。

位移量更新



相较于直接用聚合的方式计算光流，采用了逐步迭代的方式减小搜索空间更新光流。

多尺度位移量编码

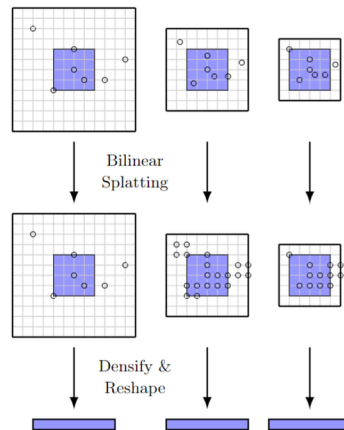
对于点 $\mathbf{x}$ ，对应的 k 个相关值集合为：

$$\{(\mathbf{d}, C(\mathbf{x}, \mathbf{d})) \mid \mathbf{d} \in S_{\mathbf{x}}^{(k)}\}$$

将坐标除以 $(1, 2, 4, 8, 16)$ ，构建5个尺度的稀疏相关体积金字塔，每一层额外通过 r 约束位移量：

$$\{(\mathbf{d}^l, C^l(\mathbf{x}, \mathbf{d}^l)) \mid \|\mathbf{d}^l\|_{\infty} \leq r, \mathbf{d} \in S_{\mathbf{x}}^{(k)}\}.$$

最后提出了一种称之为“双线性溅射(bilinear splatting)”的方法。相关值被双向分布到四个最近的整数网格上：



$$C^l(\mathbf{x}, [\mathbf{d}^l]) = (1 - |d_x^l - d_x|) (1 - |d_y^l - d_y|) C^l(\mathbf{x}, \mathbf{d}^l).$$

GRU迭代模块

与RAFT一样，这里将2D motion tensor与context特征、当前估计flow连接起来，并将其通过一个门控循环单元(GRU)更新块。GRU更新块估计残差光流，用于下一步Volumn的坐标更新。

最终实验中在不同尺寸和batch下的内存对比如下：

	Size	Memory	Size	Memory
Dense	7.8×10^8	3.1 GB	4.8×10^7	191 MB
$k = 8$	2.2×10^5	0.9 MB	5.5×10^4	0.2 MB
$k = 32$	8.9×10^5	3.6 MB	2.2×10^5	0.9 MB
$k = 128$	3.6×10^6	14.3 MB	8.8×10^5	3.5 MB

Method	batch = 1	batch = 2
RAFT [29]	10.6 GB	20.0 GB
Ours	6.1 GB	9.3 GB

客观和主观指标如下：

Training Data	Method	Sintel (train)		KITTI-15 (train)		Sintel (test)		KITTI-15 (test)	
		Clean	Final	EPE	F1-all	Clean	Final	F1-all	
C + T	LiteFlowNet2 [14]	2.24	3.78	8.97	25.9	-	-	-	-
	VCN [33]	2.21	3.68	8.36	25.1	-	-	-	-
	MaskFlowNet [36]	2.25	3.61	-	23.1	-	-	-	-
	FlowNet2 [16]	2.02	3.54	10.08	30.0	3.96	6.02	-	-
	DICL [30]	1.94	3.77	8.70	23.6	-	-	-	-
	RAFT [29]	1.43	2.71	5.04	17.4	-	-	-	-
C+T+S/K(+H)	Ours	1.29	2.95	6.80	19.3	-	-	-	-
	FlowNet2 [16]	(1.45)	(2.01)	(2.30)	(6.8)	4.16	5.74	11.48	
	PWC-Net+ [28]	(1.71)	(2.34)	(1.50)	(5.3)	3.45	4.60	7.72	
	LiteFlowNet2 [14]	(1.30)	(1.62)	(1.47)	(4.8)	3.48	4.69	7.74	
	HD3 [34]	(1.87)	(1.17)	(1.31)	(4.1)	4.79	4.67	6.55	
	IRR-PWC [15]	(1.92)	(2.51)	(1.63)	(5.3)	3.84	4.58	7.65	
	VCN [33]	(1.66)	(2.24)	(1.16)	(4.1)	2.81	4.40	6.30	
	MaskFlowNet [36]	-	-	-	-	2.52	4.17	6.10	
	ScopeFlow [1]	-	-	-	-	3.59	4.10	6.82	
	DICL [30]	(1.11)	(1.60)	(1.02)	(3.6)	2.12	3.44	6.31	
	RAFT [29] (warm-start)	(0.77)	(1.27)	-	-	1.61	2.86	-	
	RAFT [29] (2-view)	(0.76)	(1.22)	(0.63)	(1.5)	1.94	3.18	5.10	
	Ours (warm-start)	(0.86)	(1.75)	-	-	1.77	3.88	-	
	Ours (2-view)	(0.79)	(1.70)	(0.75)	(2.1)	1.72	3.60	6.17	

Input image 1

Ground-truth

RAFT [29]

Ours

3.3 DIP

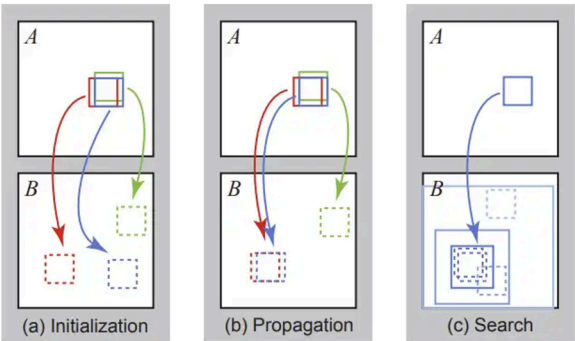
这篇文章基于RAFT，引入了DIS反向传播、PatchMatch的思想进行改良，可以有效地减少计算，保持可观的性能。DIS和PatchMatch的内容可以参照下文：

无关风月：图像对齐：传统光流算法总结

23 赞同 · 0 评论 · 文章



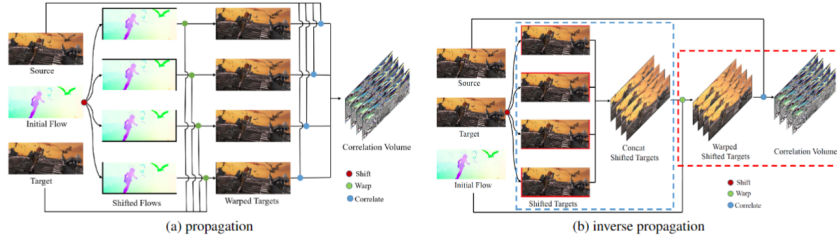
经典的相关性volume计算复杂度为 $hw(2d+1)$ ，受限搜索半径 d ，这样的相关性很难观察到全局信息，因此raft提出了全局volume。patchmatch可以通过大量的随机采样点得到一个较好的预测，在图像的局部一旦一个好的match被找到，信息可以被高效的传播到邻域。所以本文提出一个传播策略减少搜索半径，通过局部搜索提高精度。



(1) 随机初始化：通过大量的随机样本得到一些较好的估计。

(2) 传播：基于图像局部性，一旦找到一个好的匹配，信息就可以从它的邻域中有效地传播。

(3) 随机搜索：在后续传播中使用它来防止局部最优，使其在邻域中不存在好的匹配时仍能够观察到好的匹配。

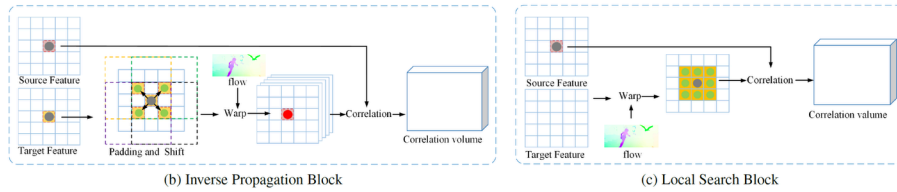


在DIP中迭代传播和搜索是解决光流问题的关键，在传播阶段把特征map中的一个点视为一个patch，并选择4个相邻的种子点，因此每个点都可以通过将光流图向4个邻域shift来从其邻域中获得光流候选点。然后就可以根据邻域光流候选点和自身的光流计算出一个5维的相关volume。给定所有光流的位移 Δp ，传播时的相关性计算可以定义为：

$$Corr = F_1 \cdot W(F_2, S(flow, \Delta p)),$$

这里S代表对flow做距离为 Δp 的shift， W 代表通过shift后的flow对 F_2 做warp。显而易见种子点越多需要的计算次数就越多，假设n个种子点做m次传播迭代，就有nxm次shift和nxm次warp。为了减少性能，我们提出了反向传播和局部搜索方法。

Deep Inverse Patchmatch



传播过程中，理论上将光流shift到右下的空间相对位置与将target移到左上是相同的，而且两种方法的相关图在绝对空间坐标上有一个像素偏移。将移动target的方式命名为反向传播，反向传播可以表述如下：

$$Corr = F_1 \cdot S(F_2', -\Delta p),$$

$$F_2' = W(S(F_2, \Delta p), flow)$$

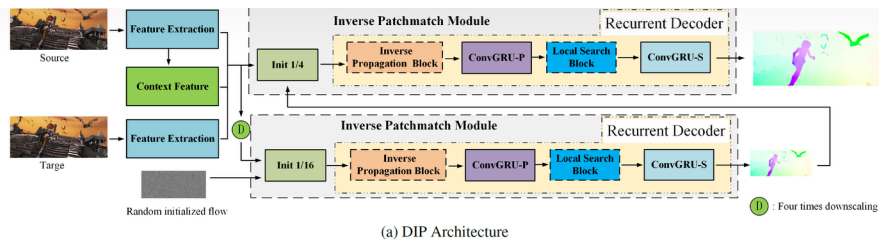
当 Δp 非常小的时候corr的计算可以近似为：

$$Corr = F_1 \cdot F_2'$$

因此我们可以提前对目标特征进行shift和stack，然后在每次迭代中只执行一次warp，就可以获得翘曲的目标特征。种子点是静态的，不会随着迭代次数的增加而改变，因此target特征只需要移动到种子点一次，并且移动的目标特征可以在每次迭代中重复使用。这样如果传播的m次迭代有n个种子点，我们只需要移动target特征n次，并将移动的目标特征warp m次。

局部搜索由于随机初始化的光流值范围非常稀疏，仅通过patch传播难以获得非常精确的光流。因此在本工作中每次patch传播后都会进行局部邻域搜索，在每次传播后执行固定的小半径搜索，称之为局部搜索。给定一个光流增量 Δf ，局部搜索可以表示为：

$$Corr = F_1 \cdot S(W(F_2, flow), \Delta f)$$



为了在高分辨率图像上获得高精度的光流，设计了一种新的光流预测框架—DIP。可以将其描述为两个主要阶段：

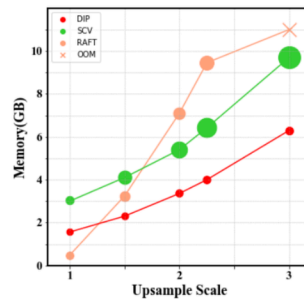
(1) 特征提取

首先，对输入图像应用特征编码器网络，提取1/4分辨率的特征图，并直接激活源特征map作为context map。然后使用Average Pooling模块将特征映射降低到1/16的分辨率。我们对1/4分辨率和1/16分辨率都使用相同的主干和参数，因此DIP可以分两个阶段进行训练，在处理大图像时，我们使用更多的阶段进行推理。

(2) 多尺度迭代更新

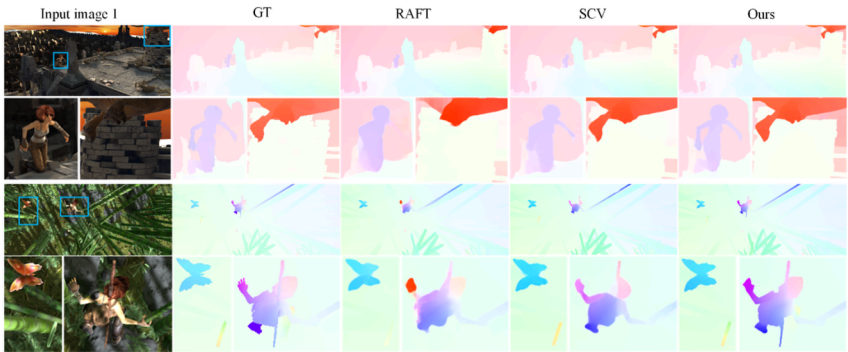
本文的方法是基于邻域传播，因此必须迭代更新光流。网络由两个模块组成，一个反向传播模块和一个局部搜索模块。在训练阶段以大小为1/16的随机光流输入网络，然后使用金字塔方法迭代优化size为1/16和size为1/4的光流。为了获得更精确的光流，我们也可以在1/8尺度下细化光流，然后在1/4尺度下优化结果。网络在推理阶段也接受初始化的光流作为输入，执行与训练阶段相同的过程。

在实验中，对比几种方案上采样到高分辨率大小的结果，其中内存限制为11GB，气泡的面积是F1-all度量的映射。我们使用KITTI数据集的上采样来评估内存和准确性，尺度为1的分辨率为375 x 1242，OOM的意思是内存不足。



客观和主观指标如下：

HD3 [55]	3.84	8.77	13.17	24.0
LiteFlowNet [21]	2.48	4.04	10.39	28.50
PWC-Net [43]	2.55	3.93	10.35	33.7
LiteFlowNet2 [22]	2.24	3.78	8.97	25.90
VCN [54]	2.21	3.68	8.36	25.10
MaskFlowNet [58]	2.25	3.61	-	23.10
FlowNet2 [23]	2.02	3.54	10.08	30
DICL [50]	1.94	3.77	8.70	23.60
RAFT [45]	1.43	2.71	<u>5.04</u>	<u>17.40</u>
Flow1D [53]	1.98	3.27	6.69	22.95
SCV [25]	1.29	2.95	6.80	19.30
ours	<u>1.30</u>	<u>2.82</u>	4.29	13.73



4.文献汇总

分类	论文名称	发表时间	代码
LiteFlowNet	LiteFlowNet: A Lightweight Convolutional Neural Network for Optical Flow Estimation	2018 CVPR	github.com/twhui/LiteFlowNet
VCN	Volumetric Correspondence Networks for Optical Flow	2019 NeurIPS	github.com/gengshan-y/VCN
LiteFlowNet2	A Lightweight Optical Flow CNN - Revisiting Data Fidelity and Regularization	2020 TPAMI	github.com/twhui/LiteFlowNet2
LiteFlowNet3	LiteFlowNet3: Resolving Correspondence Ambiguity for More Accurate Optical Flow Estimation	2020 TPAMI	github.com/twhui/LiteFlowNet3
Flow1D	High-Resolution Optical Flow from 1D Attention and Correlation	2021 ICCV	github.com/haofeixu/Flow1D
SCV	Learning Optical Flow from a Few Matches	2021 CVPR	github.com/zacjiang/SCV
DIP	DIP: Deep Inverse Patchmatch for High-Resolution Optical Flow	2022 CVPR	github.com/zihuazheng/DIP

深度学习 (Deep Learning)

赞同 11 2 条评论 分享 喜欢 收藏 申请转载 ...

写下你的评论...

2 条评论

默认 最新



Yeah



2024-10-11 · 北京

回复 1



崔星星

这里有较新的论文和代码, github.com/cuixing158/A...

03-19 · 广东

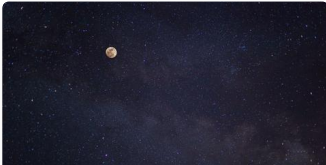
回复 喜欢

推荐阅读

揭示Transformer「周期建模」缺陷！北大提出新型神经网络...

编辑: LRST 【新智元导读】北京大学研究团队开发的FAN模型能有效捕捉数据中的周期性模式, 相比传统模型在多项任务中表现出色, 同时降低了参数量和计算量, 增强了对周期性特征的建模能力, 应...

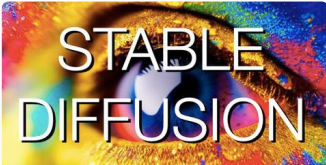
新智元



conv-LSTM解读: 背景&介绍& 优劣势&适用场景 (附生成视...

617yx

发表于机器学习与...



Stable Diffusion 万字长文详解 稳定扩散模型

7号床

发表于Stabl...

Scalar Operation

$$\begin{matrix} A_1 \times B_1 = C_1 \\ A_2 \times B_2 = C_2 \\ A_3 \times B_3 = C_3 \\ A_4 \times B_4 = C_4 \end{matrix}$$

SIMD C

$$\begin{matrix} A_1 \\ A_2 \\ A_3 \\ A_4 \end{matrix} \times \begin{matrix} B_1 \\ B_2 \\ B_3 \\ B_4 \end{matrix} = \begin{matrix} C_1 \\ C_2 \\ C_3 \\ C_4 \end{matrix}$$

工程实现 | 加速神经网络之 SIMD 向量化运算

茶多酚老爹